Git Commit Deployment Feature

Reusable Express + React implementation

This guide provides the backend routes and React page needed to load remote commit history and deploy a selected commit. Protect both endpoints with authentication and administrator authorization.

1. Backend setup

```
import { execFile } from "node:child_process";
import { promisify } from "node:util";

const execFileAsync = promisify(execFile);
app.use(express.json());

// Replace these with your application auth middleware.
app.get("/api/deploy/commits", requireAuth, requireAdmin, async (req, res) => {
  const repository = String(req.query.repository || process.env.GITHUB_REPOSITORY || "owner/repository")
    .replace(/^https?:\/\/github\.com\//, "")
    .replace(/\.git$/, "")
    .replace(/^\/+|\/+$/g, "");

  if (!/^[A-Za-z0-9_.-]+\/[A-Za-z0-9_.-]+$/.test(repository))
    return res.status(400).json({ error: "Repository must use owner/name format" });

  try {
    await execFileAsync("git", ["fetch", "--all", "--prune"], { cwd: process.cwd(), windowsHide: true });
    const { stdout } = await execFileAsync("git", [
      "log", "origin/main", "-n", "50",
      "--pretty=format:%H%x1f%an%x1f%aI%x1f%s"
    ], { cwd: process.cwd(), windowsHide: true });

    const commits = stdout.split(/\r?\n/).filter(Boolean).map(line => {
      const [fullHash, author, date, ...message] = line.split("\x1f");
      return { hash: fullHash.slice(0, 7), fullHash,
        message: message.join("\x1f"), author, date,
        url: `https://github.com/${repository}/commit/${fullHash}` };
    });
    res.json({ repository, url: `https://github.com/${repository}`, commits });
  } catch (error) {
    res.status(502).json({ error: error?.stderr || error?.message || "Unable to read Git history" });
  }
});

app.post("/api/deploy", requireAuth, requireAdmin, async (req, res) => {
  const hash = String(req.body?.hash || "").trim();
  if (!/^[0-9a-f]{7,40}$/i.test(hash))
    return res.status(400).json({ error: "A valid commit hash is required" });

  try {
    await execFileAsync("git", ["fetch", "--all", "--prune"], { cwd: process.cwd(), windowsHide: true });
    await execFileAsync("git", ["checkout", "--force", hash], { cwd: process.cwd(), windowsHide: true });
    res.json({ ok: true, commit: hash,
      message: "Commit checked out. Restart the server to load the deployed code." });
  } catch (error) {
    res.status(500).json({ error: error?.stderr || error?.message || "Deployment failed" });
  }
});
```

2. React deployment page

```
const authHeaders = () => {
  const token = localStorage.getItem("authToken");
  return token ? { Authorization: `Bearer ${token}` } : {};
};

async function loadCommits(repository) {
  const response = await fetch(
    `/api/deploy/commits?repository=${encodeURIComponent(repository)}`,
    { headers: authHeaders() }
  );
  const data = await response.json();
  if (!response.ok) throw new Error(data.error || "Unable to load commits");
  return data.commits || [];
}

async function deployCommit(hash) {
  const response = await fetch("/api/deploy", {
    method: "POST",
    headers: { "Content-Type": "application/json", ...authHeaders() },
    body: JSON.stringify({ hash })
  });
  const data = await response.json();
  if (!response.ok) throw new Error(data.error || "Deployment failed");
  return data;
}

// In the component:
const [commits, setCommits] = useState([]);
const [selected, setSelected] = useState("");
const [expanded, setExpanded] = useState(false);

useEffect(() => {
  loadCommits(repository).then(items => {
    setCommits(items);
    setSelected(items[0]?.hash || "");
  }).catch(error => setError(error.message));
}, []);

<button onClick={() => deployCommit(commits[0]?.hash)}>
  Deploy latest commit
</button>

{commits.slice(0, expanded ? commits.length : 5).map(commit => (
  <label key={commit.hash}>
    <input type="radio" checked={selected === commit.hash}
      onChange={() => setSelected(commit.hash)} />
    {commit.hash} - {commit.message} - {commit.author}
  </label>
))}

{commits.length > 5 && (
  <button onClick={() => setExpanded(value => !value)}>
    {expanded ? "Show last 5 commits" : `View all ${commits.length} commits`}
  </button>
)}

<button disabled={!selected} onClick={() => deployCommit(selected)}>
  Deploy selected commit
</button>
```

3. Environment and production notes

```
GITHUB_REPOSITORY=owner/repository
PORT=3000
```

The endpoint checks out code in the current application directory. For production, use a release directory, run the build and health checks, switch a symlink, and restart through systemd or PM2. Never expose deployment routes without admin authorization. Keep a previous release available for rollback.